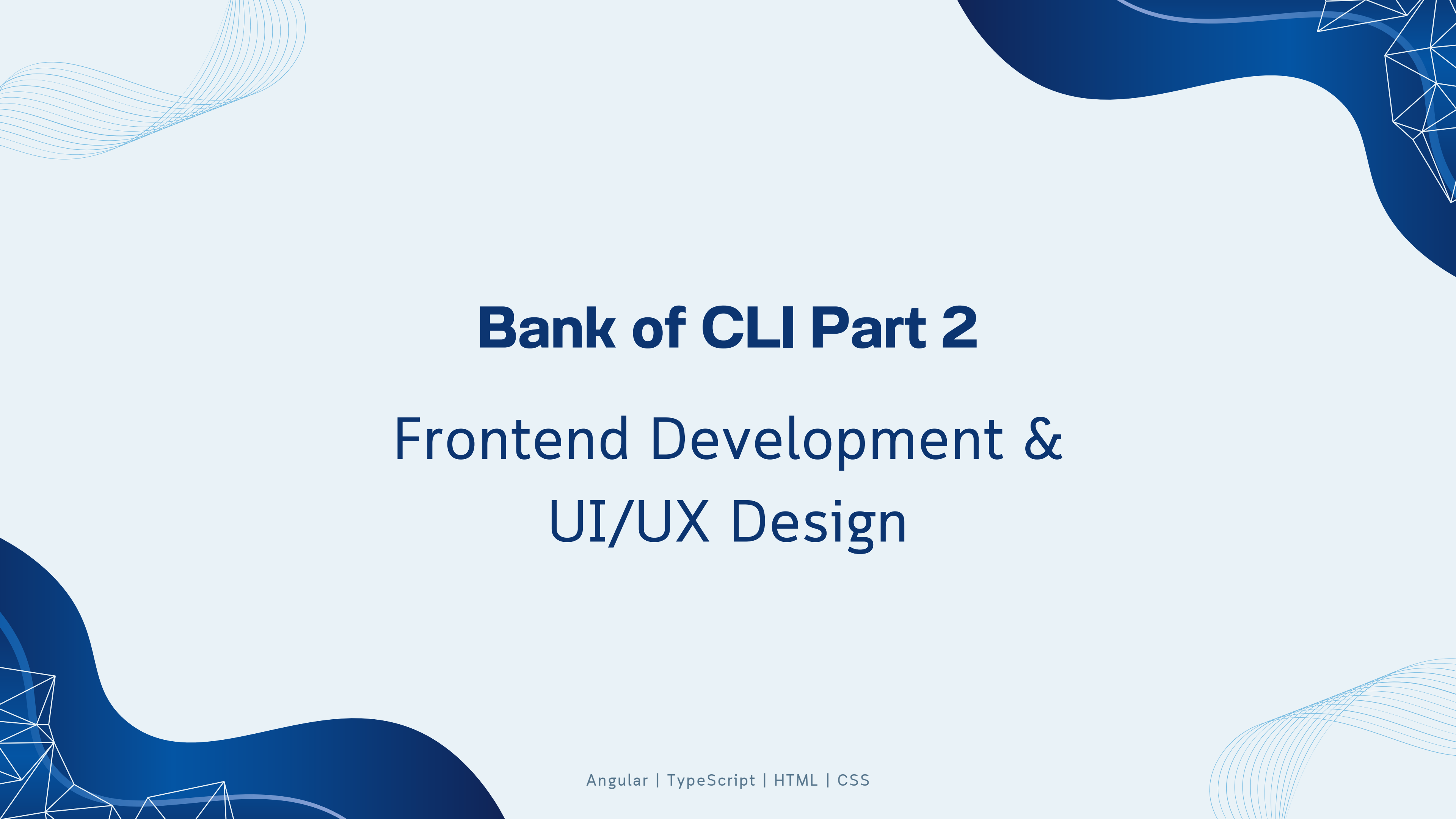




Bank of CLI Part 2

Frontend Development & UI/UX Design

Angular | TypeScript | HTML | CSS

Project Overview

Bank of CLI Part 2 transforms the terminal-based banking engine into a modern, professional single-page web dashboard. This phase focuses on the frontend experience, simulated API responses, and an API-ready architecture for the real backend to plug in later.

Project Goals

Secure Access

Create polished login and registration flows, including simulated success and invalid-credential states.

Dashboard Experience

Show the current balance and recent transactions in one central, responsive hub.

Transaction Center

Support deposit, withdrawal, and transfer forms with client-side validation, loading states, and clear feedback.

Professional UX: spinners or skeleton loaders, toast notifications, and error states make every action understandable.

Tech Stack

Framework



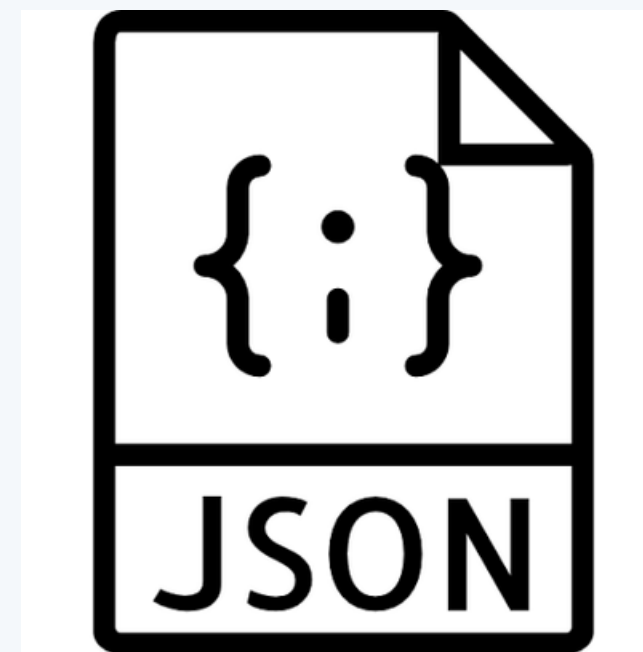
Language



Styling



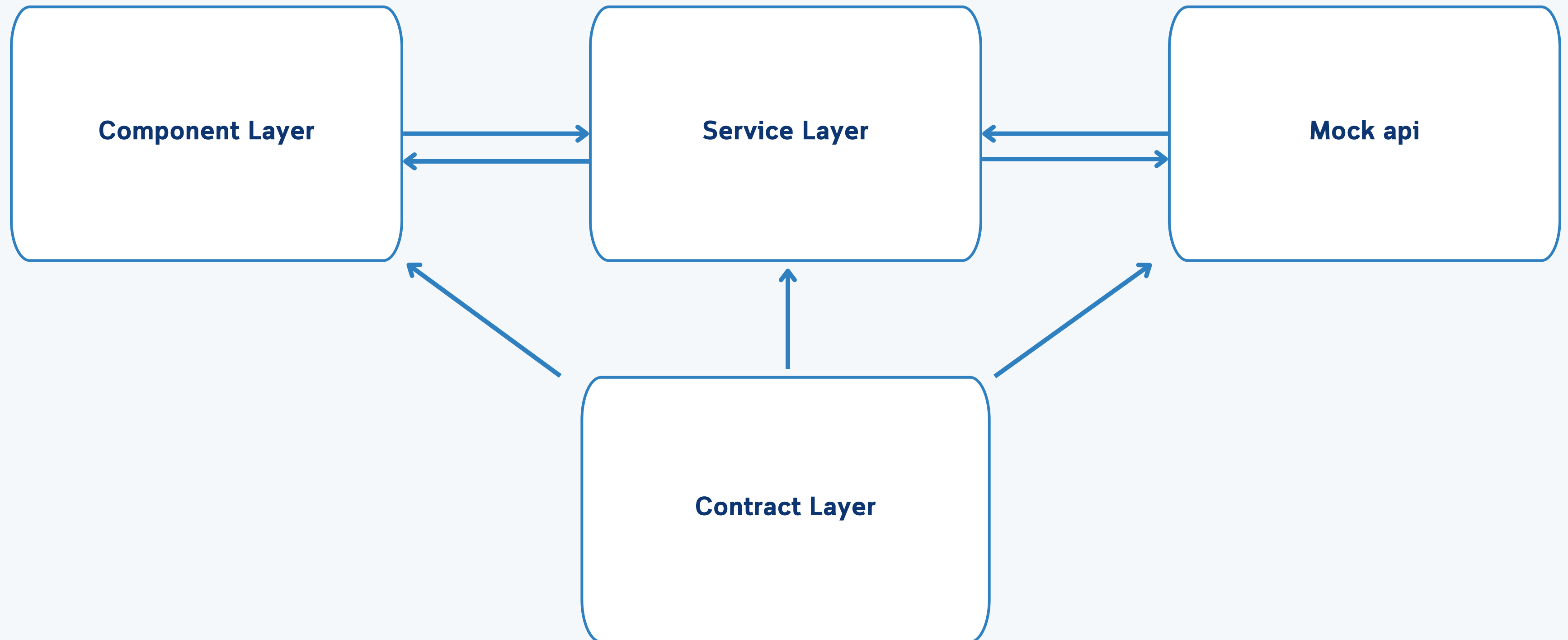
Data simulation



Version control



System Architecture



Component Layer

- The component layer is what the user sees and interacts with.
- We split the UI into separate components so team members could work on different parts of the application independently.
- Feature components include Login, Account Overview, Analytics, Transactions, and Transaction History.
- Each component has its own responsibility, which keeps the code easier to understand and maintain.
- Components get the data they need through services instead of directly accessing the mock data, which keeps the UI separate from the data logic and makes the application easier to maintain or change later.

Service Layer

What the Service Layer Handles

- Validation: Amounts must be positive with at most 2 decimals, checks funds and recipient.
- Live updates: After a transaction, the balance and history refresh everywhere automatically.
- Loading state: One isLoading flag drives every spinner.
- Consistent errors: One error type with codes like INSUFFICIENT_FUNDS and ACCOUNT_NOT_FOUND.
- Clean data: Passwords never reach the UI and timestamps are normalized for every browser.

Contract Layer

Defining data structures between the frontend and API.

TypeScript: firstName: string;

JSON: "first_name": "John"

Account: account details, ownership, and balance

Transaction: type, amount, description, timestamp, and participants

Ensures consistent data structures across the Angular application.

Mock Data/API

The **Mock API** reads **Mock Data** and allows the service layer to communicate with that data. The API does not manipulate any JSON data.

✓ mock

{ } account.json

TS api.ts

{ } transaction.json

```
> export function addAccount(account: Account): Account { ...  
  }  
  
> export function getAccounts() { ...  
  }  
  
> export function getAccountById(id: number) { ...  
  }  
  
> export function getTransactionsByAccountId(id: number) { ...  
  }  
  
> export function deposit(amount: number, accountId: number) { ...  
  }  
  
> export function withdraw(amount: number, accountId: number) { ...  
  }  
  
> export function transfer(amount: number, accountId: number, relatedAccountId: number) { ...  
  }
```

Routing

- Angular's router manages client-side requests
- Dynamically updates page contents without reloading the page (SPA)

Our Pages

/

landing page

/login

login and registration

/dashboard

protected by an auth guard

/**

not-found page for anything unmatched

app.html

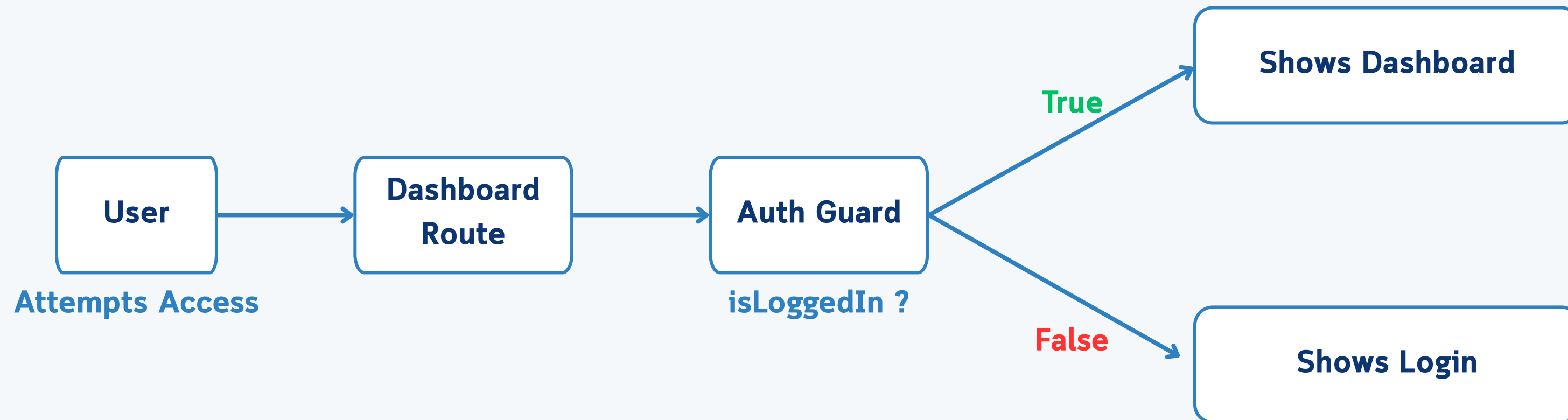
```
<app-navbar></app-navbar>
<router-outlet></router-outlet>
```

app-routes.ts

```
export const routes: Routes = [
  {
    path: "login",
    component: Login
  },
  {
    path: "dashboard",
    component: Dashboard,
    canActivate: [authenticationGuard]
  },
  {
    path: "",
    component: Home,
    pathMatch: "full" // Ensures full p
  },
  {
    path: "**",
    component: NotFound
  }
];
```

Guards

- Prior to giving the user access to a route, A route protected with a guard checks if the user meets a certain condition before accessing a route.
- The Dashboard route is protected via the Authentication Guard.
- The Authentication Guard checks with the User Service layer to ensure the user is authenticated before giving access to the Dashboard route.
- If an unauthenticated user tries to access the Dashboard, they are redirected to the Login page



DEMO!

Q & A

Bank of CLI Part 2 · Frontend dashboard experience